



CLOUDPANDITH ONLINE TRAININGS



Master Data Engineering with Expert-Led Live Training

◇ Azure Data Factory

Complete ETL/ELT

◇ Azure Databricks

Big Data & PySpark

◇ Microsoft Fabric

End-to-End Analytics

📞 ENROLL: **8904424822** | 🌐 www.cloudpandith.com

✓ Live Sessions ✓ Real Projects ✓ Expert Trainers ✓ Interview Prep ✓ Lifetime Access

- ◆ Live Classes
- ◆ Real-Time Projects
- ◆ Interview Preparation
- ◆ Lifetime Access

Index

1. Introduction to Schema Changes in Delta Lake
2. Schema Enforcement
3. Schema Evolution
4. Schema Enforcement vs Schema Evolution
5. Time Travel
6. Vacuum
7. Parquet File Cleanup vs Log Cleanup
8. Retention Summary
9. Complete Delta Lake Flow
10. Interview Questions and Answers

1. Schema Enforcement | Schema Evolution | Time Travel | Vacuum

How to Handle Schema Changes Dynamically Using Delta Lake Features

What is Schema in Delta Lake?

Schema means the structure of the data.

Example:

emp_id	emp_name	salary
101	John	5000

Schema:

emp_id -> integer
emp_name -> string
salary -> integer

Schema Enforcement

What is Schema Enforcement?

Schema Enforcement means Delta Lake validates incoming data schema before writing into the table.

If schema does not match existing table schema, Delta Lake throws an error.

It helps maintain data quality and prevents bad data from entering the table.

Existing Delta Table

```
from pyspark.sql.types import *

schema = StructType([
    StructField("emp_id", IntegerType()),
    StructField("emp_name", StringType()),
    StructField("salary", IntegerType())
])

df = spark.createDataFrame([
    (101, "John", 5000)
], schema)

df.write.format("delta").save("/mnt/delta/employee")
```

New Data with Different Schema

```
new_df = spark.createDataFrame([
    (102, "Mike", 6000, "USA")
], ["emp_id", "emp_name", "salary", "country"])
```

New column:

country

Writing Data Without Schema Evolution

```
new_df.write.format("delta").mode("append").save("/mnt/delta/employee")
```

Result

Delta Lake throws error:

AnalysisException:

A schema mismatch detected while writing to the Delta table.

Why Schema Enforcement is Important?

- Prevents invalid columns

- Prevents wrong datatype insertion
 - Maintains table consistency
 - Protects downstream pipelines
 - Improves data governance
-

2. Schema Evolution

What is Schema Evolution?

Schema Evolution allows Delta Lake to automatically modify table schema when new columns arrive.

Instead of failing the job, Delta Lake updates the schema dynamically.

Example

Existing Table:

emp_id emp_name salary

Incoming Data:

emp_id emp_name salary country

New column:

country

Enable Schema Evolution

Method 1 : mergeSchema

```
new_df.write \  
  .format("delta") \  
  .option("mergeSchema", "true") \  
  .mode("append") \  
  .save("/mnt/delta/employee")
```

Result

Table becomes:

emp_id	emp_name	salary	country
101	John	5000	null
102	Mike	6000	USA

Auto Schema Evolution

Enable globally:

```
spark.conf.set(  
    "spark.databricks.delta.schema.autoMerge.enabled",  
    "true"  
)
```

Now Delta automatically evolves schema during merge/update operations.

Real-Time Use Cases

- APIs adding new fields
- IoT devices sending additional columns
- E-commerce applications changing payload structure
- Streaming data schema changes
- Slowly evolving business systems

Schema Enforcement vs Schema Evolution

Feature	Schema Enforcement	Schema Evolution
Purpose	Prevent invalid schema	Allow schema changes
Behavior	Throws error	Updates schema
Data Quality	Strict	Flexible
Usage	Production governance	Dynamic ingestion
Default	Enabled	Disabled

3. Time Travel

What is Time Travel?

Time Travel allows reading older versions of Delta tables.

Delta Lake maintains transaction history in delta logs.

You can query previous versions even after updates or deletes.

Why Time Travel?

- Recover deleted data
- Audit historical changes
- Debug pipelines
- Compare old vs new data
- Reprocess historical data

Example

Version 0

```
df.write.format("delta").save("/mnt/delta/employee")
```

Version 1

```
new_df.write.format("delta").mode("append").save("/mnt/delta/employee")
```

Read Current Data

```
spark.read.format("delta").load("/mnt/delta/employee")
```

Read Older Version

Using Version Number

```
spark.read.format("delta") \
  .option("versionAsOf", 0) \
  .load("/mnt/delta/employee")
```

Using Timestamp

```
spark.read.format("delta") \
  .option("timestampAsOf", "2026-05-01") \
  .load("/mnt/delta/employee")
```

View History

```
from delta.tables import DeltaTable
```

```
deltaTable = DeltaTable.forPath(
    spark,
    "/mnt/delta/employee")
```

```
deltaTable.history().show()
```

Internally How Time Travel Works

Delta Lake stores:

- JSON transaction logs
- Checkpoint parquet files
- Old parquet data files

Inside:

`_delta_log`

Directory.

4. Vacuum

What is Vacuum?

Vacuum removes old unused parquet files from Delta Lake storage.

When update/delete/merge happens:

- Old files are not immediately deleted
- Delta marks them as inactive
- Vacuum permanently removes them later

Why Vacuum?

- Reduce storage usage
- Remove unnecessary old files
- Improve storage management

Important Retention Rules

Default Retention

7 days

Default vacuum retention period:

168 hours

Old parquet files older than 7 days are deleted.

Vacuum Example

VACUUM employee

or

```
spark.sql("VACUUM delta.`/mnt/delta/employee`")
```

Vacuum with 30 Days

VACUUM employee RETAIN 720 HOURS

720 hours = 30 days

Vacuum Immediately

```
spark.conf.set(  
  "spark.databricks.delta.retentionDurationCheck.enabled",  
  "false"  
)
```

VACUUM employee RETAIN 0 HOURS

Important Note

Immediate vacuum is dangerous because:

- Time travel stops working
- Old versions cannot be recovered
- Streaming jobs may fail

Usually used only in:

- Development
 - Testing
 - Temporary environments
-

Parquet File Cleanup vs Log Cleanup

Parquet File Cleanup

Handled by: VACUUM

Default: 7 days

Old unused parquet files deleted after 7 days.

Log Cleanup

Delta transaction logs cleaned separately.

Default: 30 days

Controls: delta.logRetentionDuration

Example:

```
ALTER TABLE employee
SET TBLPROPERTIES (
  'delta.logRetentionDuration' = '30 days'
)
```

Retention Summary

Component	Default Retention
Old Parquet Files	7 days
Delta Transaction Logs	30 days
Time Travel Availability	Depends on retained files/logs

Complete Flow

New Data Arrives
↓
Schema Enforcement validates schema
↓
Schema Evolution handles new columns
↓
Delta Logs maintain table versions
↓
Time Travel accesses old versions
↓
Vacuum removes old unused parquet files

Interview Questions

1. What is Schema Enforcement?

Prevents invalid schema writes into Delta tables.

2. What is Schema Evolution?

Allows automatic schema updates when new columns arrive.

3. Difference between Enforcement and Evolution?

Enforcement blocks mismatched schema.

Evolution accepts and updates schema.

4. What is Time Travel?

Reading historical versions of Delta tables.

5. What is Vacuum?

Deletes old unused parquet files.

6. Default Vacuum Retention?

7 days

7. Default Delta Log Retention?

30 days

8. Why Immediate Vacuum is Dangerous?

Removes old files permanently and breaks time travel.